

# SD Basic - Terminal Input and Output

Writing to the screen, reading from the keyboard, and positioning the cursor.

---

This page covers what a program says to the person running it and what it reads back: the output statements, the input statements, and the cursor and key handling underneath them.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

---

## Output

```
print {on print.unit} expression {:}  
crt expression {:}  
display expression {:}
```

All three write a line. The differences matter only once a print unit is involved:

---

|                      |                                                                                                          |
|----------------------|----------------------------------------------------------------------------------------------------------|
| <code>print</code>   | goes to the <b>current print unit</b> — the terminal normally, the printer after <code>printer on</code> |
| <code>crt</code>     | <b>always</b> goes to the terminal, even while <code>printer on</code> is in force                       |
| <code>display</code> | a synonym for <code>crt</code>                                                                           |

---

**That is the whole reason `crt` exists.** A report that prints its output and shows progress on screen uses `print` for the report and `crt` for the progress; using `print` for both puts "Processing record 400..." in the middle of the printed report.

**A trailing colon suppresses the line ending**, so the next statement continues on the same line:

```
crt 'Working' :  
for i = 1 to 10  
    crt '.' :  
next i  
crt ''
```

## LAYING OUTPUT OUT

The format qualifier goes straight after the value, with nothing between them:

```
print total '10r2'
```

`42 'R#8'` gives `42` — right-justified in eight columns. The full specification language is in SD Basic - Data Conversion, along with the warning that a value longer than the width **wraps** rather than truncating.

---

## The cursor

```
@(column, row)
@(code)
```

`@()` returns a string of terminal control characters. Printing it moves the cursor or changes the display; it is not a statement.

**On this port it emits ansi escape sequences.** `@(0,0)` is six bytes — `27 91 49 59 49 72`, which is `ESC [ 1 ; 1 H`, the standard cursor positioning sequence. `@(-1)`, clear screen, is also six bytes.

### CODE

|                                           |                                      |
|-------------------------------------------|--------------------------------------|
| <code>@(-1)</code>                        | clear the screen and home the cursor |
| <code>@(-2)</code>                        | home the cursor without clearing     |
| <code>@(-3)</code>                        | clear to end of screen               |
| <code>@(-4)</code>                        | clear to end of line                 |
| <code>@(-5)</code> · <code>@(-6)</code>   | cursor off, cursor on                |
| <code>@(-13)</code> · <code>@(-14)</code> | reverse video on, off                |

**The sequences come from the terminal definition, not from a fixed table.** A session whose terminal type is wrong gets sequences the terminal does not understand and displays as literal text. `term` at the command line reports and sets the type.

---

## Screen geometry

```
@crtwide
@crthigh
terminfo(key)
```

`@crtwide` and `@crthigh` are the current width and height. After `term 200,9999` they read **200** and **9999** — they follow whatever `term` last set, which is why a program must read them rather than assume 80 by 24.

`terminfo()` reports a named capability of the terminal definition. In a session with **no terminal**, `terminfo('name')` and `terminfo('cols')` both return the **null string** — so a program that formats a screen from `terminfo()` must cope with getting nothing back when it is run from a script or the API.

```
save.screen(column, row, width, height)
restore.screen(saved, column, row)
```

save and restore a rectangle, for a program that pops a window over what is already displayed.

---

## Input

```
input variable {, length} {:} {with prompt} {format} {then ... else ...}
input @(column, row) {, length} : variable
inputfield variable, length {, ...}
```

`input` reads a line. The optional *length* limits what may be typed and returns as soon as that many characters arrive, without waiting for Enter.

### CLAUSE

|                |                                                                                |
|----------------|--------------------------------------------------------------------------------|
| <b>append</b>  | start with the variable's current value, so the user edits rather than retypes |
| <b>overlay</b> | as append, but typing replaces from the first character                        |
| <b>edit</b>    | full line editing                                                              |
| <b>hidden</b>  | do not echo — for a password                                                   |
| <b>panning</b> | scroll a field wider than the space it is shown in                             |
| <b>upcase</b>  | fold what is typed to upper case                                               |

---

## CLAUSE

---

**timeout**  $n$  give up after  $n$  seconds and take the `else` branch

---

`hidden` **is the only correct way to read a password.** Turning the echo off by hand with `echo off` leaves it off if the program aborts between the two statements, and the user then types blind at the command prompt.

```
inputerr message
inputclear
cleardata
data expression
```

---

|                       |                                                                 |
|-----------------------|-----------------------------------------------------------------|
| <code>inputerr</code> | show a message on the status line without disturbing the screen |
|-----------------------|-----------------------------------------------------------------|

---

|                         |                              |
|-------------------------|------------------------------|
| <code>inputclear</code> | discard anything typed ahead |
|-------------------------|------------------------------|

---

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <code>data</code> | <b>stack</b> a value for the next <code>input</code> to consume |
|-------------------|-----------------------------------------------------------------|

---

|                        |                                        |
|------------------------|----------------------------------------|
| <code>cleardata</code> | discard what <code>data</code> stacked |
|------------------------|----------------------------------------|

---

`data` is how one program answers another's prompts:

```
data 'Y'
execute 'DELETE.FILE OLDSTUFF'
```

**A `data` value that is not consumed stays stacked.** The next `input` anywhere in the session takes it — including a prompt the user was meant to answer. `cleardata` **after anything that might not have consumed it.**

---

## Keys

```
keyin()
keyinc()
keyinr()
keyready()
keycode()
```

---

|                         |                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------|
| <code>keyin()</code>    | wait for one keystroke and return it                                              |
| <code>keyinc()</code>   | as <code>keyin()</code> , but translate an escape sequence into a single key code |
| <code>keyinr()</code>   | raw — no translation at all                                                       |
| <code>keyready()</code> | true if a keystroke is waiting; does not consume it                               |
| <code>keycode()</code>  | the code of the key that ended the last <code>input</code>                        |

---

```
keyedit code, key.sequence
keyexit code, key.sequence
keytrap code, key.sequence
bindkey(key.string, action)
```

These attach a key to an editing action, an exit from `input`, or a trap. `keycode()` then tells the program which one fired, so a form can act on F3 without reading raw escape sequences.

---

## Prompts and headings

```
prompt character
heading {no.eject} {on print.unit} text
footing {on print.unit} text
page {on print.unit} {page.number}
```

`prompt` sets the character SD shows when it wants input — a colon by default; `prompt ''` suppresses it.

`heading` and `footing` are covered with the rest of the paging machinery in SD Basic - Printing, because they apply to the terminal and the printer in the same way.

---

## Echo and silence

```
echo on | off | expression
hush on | off | expression
```

---

|                   |                                 |
|-------------------|---------------------------------|
| <code>echo</code> | whether typed characters appear |
| <code>hush</code> | whether <b>output</b> appears   |

---

`hush on` suppresses output entirely — used around an `execute` whose chatter you do not want. Both nest badly: they are a session setting, not a stack, so a program that aborts between `hush on` and `hush off` leaves the session silent and the user with no way to know why.

---

## Reading the command line

`tclread variable`

Returns the command line that started the program, so a verb written in BASIC can parse its own arguments. `@sentence` holds the same thing.

---

## The port's terminal, in one paragraph

SD Core for Windows turns on the console's ANSI processing itself rather than inheriting it, so `@()` sequences work in an ordinary PowerShell window without the user configuring anything. Over ssh the session is a console session, so the same sequences reach the client terminal. **SD is a terminal system throughout; the API is the only way to reach it without one**, and there a program is reading a command's output rather than driving a screen. **Such a session has no geometry and no capabilities**, which is why `terminfo()` returns the null string above rather than guessing.

---

## What is not here

---

|                                             |                                                                                  |
|---------------------------------------------|----------------------------------------------------------------------------------|
| <code>ttyget()</code> · <code>ttyset</code> | <b>removed</b> — they exposed a POSIX terminal structure this port does not have |
| <code>connect.port()</code>                 | <b>removed</b>                                                                   |

---

## See also

SD Basic - Printing · SD Basic - Data Conversion · SD Basic - Program Control.

